This example also adds some extra information about each step. The texts following the `given`, `when`, and `then` labels are optional, but they're often used to make the intent of the specification clearer. Sometimes a bit of extra explanation goes a long way. Other times the code itself is sufficient.

Spock also supports a more lightweight syntax that's useful for example-based testing. For example, you could describe the minimum points required to upgrade to the next status level like this:

```
class WhenCheckingMinimumStatusPoints extends Specification {

    def "should know the minimum points required for each status level"() {
        expect:
            FrequentFlyerStatus.statusLevelFor(points) == expectedStatus
        where:
            points   | expectedStatus
            0        | BRONZE
            299      | BRONZE
            300      | SILVER
            699      | SILVER
            700      | GOLD
            1499     | GOLD
            1500     | PLATINUM
    }
}
```

**The status level obtained with a given number of points**

**The expected statuses for a range of point values**

The approach used by tools like Spock and Specs2 hits a sweet spot between expressive executable requirements and technical documentation. When using tools like Cucumber or JBehave, you keep the feature files separate from the step definitions, since the scenarios are discussed, defined, and owned by the team as a whole. Adding implementation code to the mix would make these feature files harder for non-developers to understand. But the more granular, low-level executable requirements we've looked at in this chapter need to combine technical documentation, business justification, and sample code.

## 10.5 Using executable specifications as living documentation

From a BDD perspective, writing a good unit test is an exercise in good communication. When you practice BDD, you think of every unit test as a low-level specification that illustrates some aspect of how a class or component behaves. You've seen ways to help ensure that the intent of these low-level specifications is clearer. But the implementation of your test is also sample code that illustrates how a particular requirement is satisfied, or how a particular goal is achieved. The code inside your tests doesn't just exercise the application; it documents *how to exercise* the application.

Not keeping your test code clean and easy to understand has very practical consequences. If an old test fails when you make a change elsewhere in the code base, it's essential to understand both what the test was trying to demonstrate and how it was doing so. Only by understanding this will you be in a position to decide what to do with the failing test.